

services\data\api_client.py

```
import requests
import json
import time
from datetime import datetime, timezone, timedelta
from typing import List, Dict, Any, Optional
import config
import threading
import logging
import gc

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class NVDApiClient:
    """Robust NVD API client with memory-safe caching and thread-safe access"""

    def __init__(self):
        self.base_url = config.NVD_API_URL
        self.api_key = config.NVD_API_KEY
        self.timeout = config.API_TIMEOUT
        self.last_request_time = 0

    # Rate limiting
    if self.api_key:
        self.min_request_interval = 30 / 50 # 50 requests per 30 seconds with key
    else:
        self.min_request_interval = 30 / 5 # 5 requests per 30 seconds without key

    # HTTP session
    self.session = requests.Session()
    if self.api_key:
        self.session.headers.update({"apiKey": self.api_key})

    # Lightweight thread-safe cache
    self._cache_timestamps = {}
    self._cache_lock = threading.Lock()

    # Database manager
    from database import db_manager
    self.db = db_manager

    # Cache manager
    from services.cache.cache_manager import cache_manager
    self.cache_manager = cache_manager

    logger.info("[API Client] Initialized with API key: %s", bool(self.api_key))

    def _rate_limit(self):
        """Ensure proper rate limiting"""
        elapsed = time.time() - self.last_request_time
        if elapsed < self.min_request_interval:
            sleep_time = self.min_request_interval - elapsed
            logger.debug("Rate limiting - sleeping for %.2f seconds", sleep_time)
            time.sleep(sleep_time)
        self.last_request_time = time.time()

    def _is_cache_valid(self, cache_key: str, max_age: int = 900) -> bool:
        """Check if cache entry is valid"""
        with self._cache_lock:
            ts = self._cache_timestamps.get(cache_key)
            if ts is None:
                return False
            return (time.time() - ts) < max_age

    def _set_cache_timestamp(self, cache_key: str):
        """Update cache timestamp"""
        with self._cache_lock:
            self._cache_timestamps[cache_key] = time.time()

    def get_cves_last_30_days(self) -> List[Dict[str, Any]]:
        """Fetch CVEs from the last 30 days, memory-safe and thread-safe"""
```

```

cache_key = "cves_30_days"

# Check in-memory cache first
cached = self.cache_manager.get_api_data_30_days()
if isinstance(cached, list):
    logger.info("[API Client] Using cached CVE data (%d CVEs)", len(cached))
    if self.db.use_database:
        # Optionally fetch a small batch from DB
        return self.db.get_cves_by_filter(limit=2000)
    return cached
else:
    if cached is not None:
        logger.warning("[API Client] Cache returned unexpected type (%s) - ignoring cached value", type(cached))

    logger.info("[API Client] Fetching fresh CVE data from API")
    end_date = datetime.now(timezone.utc)
    start_date = end_date - timedelta(days=30)
    all_cves = []
    start_index = 0
    results_per_page = 2000

    try:
        while True:
            self.cache_manager.check_memory_usage()

            params = {
                "resultsPerPage": results_per_page,
                "startIndex": start_index,
                "pubStartDate": start_date.strftime("%Y-%m-%dT%H:%M:%S.000"),
                "pubEndDate": end_date.strftime("%Y-%m-%dT%H:%M:%S.000")
            }

            logger.info("API request: startIndex=%d, resultsPerPage=%d", start_index, results_per_page)
            self._rate_limit()
            response = self.session.get(self.base_url, params=params, timeout=self.timeout)

            if response.status_code != 200:
                logger.error("API request failed: %s - %s", response.status_code, response.text)
                break

            data = response.json()
            vulnerabilities = data.get("vulnerabilities", [])
            total_results = data.get("totalResults", 0)
            logger.info("Got %d CVEs (total available: %d)", len(vulnerabilities), total_results)

            if not vulnerabilities:
                break

            batch_processed = []
            for item in vulnerabilities:
                cve_data = item.get("cve", {})
                processed = self._process_cve_item(cve_data)
                if processed:
                    batch_processed.append(processed)

            # Save batch to DB
            if batch_processed and self.db.use_database:
                self.db.save_cves_batch(batch_processed)

            all_cves.extend(batch_processed)
            gc.collect()

            start_index += results_per_page
            if start_index >= total_results:
                break

            if not all_cves:
                logger.warning("[API Client] No CVEs processed from API")
            else:
                logger.info("[API Client] Successfully processed %d CVEs", len(all_cves))

            all_cves.sort(key=lambda x: x.get('Published', ''), reverse=True)

            # Cache in memory
            self.cache_manager.set_api_data_30_days(all_cves)

```

```

self._set_cache_timestamp(cache_key)

return all_cves

except Exception as e:
    logger.error("Error fetching CVEs: %s", e)
    if self.db.use_database:
        logger.info("[API Client] Falling back to database")
        return self.db.get_all_cves(limit=1000)
    return []

def _process_cve_item(self, cve_data: Dict) -> Optional[Dict]:
    """Convert NVD CVE format to internal memory-safe format"""
    try:
        if not cve_data:
            return None

        cve_id = cve_data.get("id")
        if not cve_id:
            return None

        description = ""
        for desc in cve_data.get("descriptions", []):
            if desc.get("lang") == "en":
                description = desc.get("value", "")
            break

        severity = "UNKNOWN"
        cvss_score = None
        metrics = cve_data.get("metrics", {})

        for version in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2"]:
            metric_list = metrics.get(version)
            if metric_list:
                try:
                    metric = metric_list[0]
                    cvss_data = metric.get("cvssData", {})
                    severity = cvss_data.get("baseSeverity", "UNKNOWN").upper()
                    cvss_score = cvss_data.get("baseScore")
                break
            except Exception:
                continue

        cwe = None
        for weakness in cve_data.get("weaknesses", []):
            for desc in weakness.get("description", []):
                if desc.get("lang") == "en":
                    val = desc.get("value")
                    if val and val.startswith("CWE"):
                        cwe = val
            break
        if cwe:
            break

        references = [ref["url"] for ref in cve_data.get("references", [])[:5] if "url" in ref]

        return {
            "ID": cve_id,
            "Description": description or "No description available",
            "Severity": severity,
            "CVSS_Score": cvss_score,
            "CWE": cwe,
            "Published": cve_data.get("published", ""),
            "lastModified": cve_data.get("lastModified", ""),
            "References": references,
            "Products": [],
            "metrics": {}
        }
    except Exception as e:
        logger.error("Error processing CVE %s: %s", cve_data.get('id', 'unknown'), e)
        return None

def get_cve_detail(self, cve_id: str) -> Dict[str, Any]:
    """Get details for a specific CVE"""
    try:
        if self.db.use_database:
            db_cve = self.db.get_cve_detail(cve_id)

```

```

if db_cve:
    return db_cve

cached_cves = self.get_cves_last_30_days()
if not isinstance(cached_cves, list):
    cached_cves = []

match = next((cve for cve in cached_cves if cve['ID'] == cve_id), None)
if match:
    return match

logger.info("Fetching CVE %s from API", cve_id)
self._rate_limit()
response = self.session.get(f"{self.base_url}?cveId={cve_id}", timeout=self.timeout)
if response.status_code != 200:
    return self._generate_fallback_cve(cve_id, f"Status {response.status_code}")

data = response.json()
vulns = data.get("vulnerabilities", [])
if not vulns:
    return self._generate_fallback_cve(cve_id, "No vulnerability data")

processed = self._process_cve_item(vulns[0].get("cve", {}))
if not processed:
    return self._generate_fallback_cve(cve_id, "Processing failed")

if self.db.use_database:
    self.db.save_cve_detail(processed)

return processed
except Exception as e:
    logger.error("Error getting CVE detail for %s: %s", cve_id, e)
    return self._generate_fallback_cve(cve_id, str(e))

def _generate_fallback_cve(self, cve_id: str, error_message: str = "") -> Dict[str, Any]:
    """Return fallback CVE structure"""
    return {
        "ID": cve_id,
        "Description": f"No details available for {cve_id}. {error_message}",
        "Severity": "UNKNOWN",
        "CVSS_Score": None,
        "CWE": "Unknown",
        "Published": "",
        "lastModified": "",
        "References": [],
        "Products": [],
        "metrics": {},
        "Error": error_message
    }

def clear_cache(self):
    """Clear all caches"""
    with self._cache_lock:
        self._cache_timestamps.clear()
        if self.db.use_database:
            self.db.clear_all_cves()
        self.cache_manager.clear_all()
    logger.info("API client cache cleared")

def get_cache_stats(self) -> Dict[str, Any]:
    """Return cache statistics"""
    with self._cache_lock:
        stats = {
            "cached_entries": len(self._cache_timestamps),
            "cache_ages": {k: f"{int((time.time()-v)/60)} minutes" for k,v in
                self._cache_timestamps.items()}
        }
    if self.db.use_database:
        stats["database"] = self.db.get_stats()
    return stats

# Global instance
api_client = NVDApiClient()

```